6. Estructuras de repetición

En programación, la **iteración** es el proceso de repetir una serie de instrucciones múltiples veces hasta que se cumpla una condición específica. Esta característica es fundamental porque permite automatizar tareas repetitivas, optimizar el código y reducir la cantidad de líneas necesarias para resolver problemas.

Existen dos tipos principales de iteración:

- **Iteración con un número determinado de repeticiones**: Se usa cuando sabemos de antemano cuántas veces queremos repetir un bloque de código. En Python, esto se logra principalmente con el bucle **for.**
- **Iteración con una condición de control**: En este caso, la repetición continúa mientras se cumpla una condición lógica. En Python, este tipo de iteración se implementa con el bucle **while.**

Además, el uso de iteraciones permite evitar código redundante y mejorar la eficiencia de los programas.

6.1. Ciclo while

El bucle **while** en Python permite ejecutar un bloque de código **mientras se cumpla una condición lógica**. Esto significa que la cantidad de iteraciones no está definida de antemano, sino que depende de la evaluación de una condición booleana.

```
contador = 1
while contador <= 5:
    print("Vuelta número", contador)
    contador += 1
print("Fin del programa")
```

1. La variable **contador** inicia en 1.
2. Mientras **contador** sea menor o igual a 5, se ejecuta el **print().**
3. En cada iteración, se incrementa **contador.**
4. Cuando **contador** es 6, la condición **contador <= 5** es **falsa** y el bucle se detiene.

Salida por pantalla

```
Vuelta número 1
Vuelta número 2
Vuelta número 3
Vuelta número 4
Vuelta número 5
Fin del programa
```

6.1.1 ¡Cuidado con los bucles infinitos!

Si la condición del **while nunca se vuelve falsa**, el bucle se ejecutará indefinidamente, causando que el programa no termine.
Por ejemplo, el siguiente código nunca se detiene porque **contador** nunca cambia:

```
contador = 1
while contador <= 5:
    print("Vuelta número", contador)
print("Fin del programa")
```

6.1.2. Ciclo while con Entrada del Usuario

Este programa pide números positivos y los suma, cortando el ciclo cuando se ingresa un número negativo, al final muestra el resultado.

```
suma = 0
numero = int(input("Ingrese un número positivo(0 negativo para salir): "))

while numero >= 0:
    suma += numero
    numero = int(input("Ingrese otro número positivo: "))

print("La suma total es:", suma)
```

Como observamos, hay una instrucción para ingreso de datos, habilita al ciclo para que empiece a iterar y antes de que el ciclo termine la vuelta vuelve a repetir la instrucción, si esta última no estuviera sería imposible que el ciclo siga iterando ya que el programa no vuelve a ejecutar la institución que dio inicio.

6.2. Ciclo for

El bucle **for** permite repetir una acción un número determinado de veces. A diferencia de **while**, aquí sabemos de antemano cuántas veces se repetirá el código.

```
for variable in range (valor_inicial, valor_final):
        #línea de código
        #línea de código
        #línea de código
```

- **valor_inicial** es el número desde donde empieza.
- **valor_final** es el número hasta donde termina (**no se incluye**).

Salvo que tengamos un programa específico, se recomienda iniciar el ciclo en 0 ya que en programación empezamos a contar desde el mismo.

```
for x in range (0, 5):
        print("Repetición número", x + 1)

print("Fin del programa")
```

Si comenzamos desde el 0, no es necesario especificarlo, Python si no encuentra ese valor toma que se empieza desde el 0:

```
for x in range (5):
        print("Repetición número", x + 1)

print("Fin del programa")
```

Si iniciamos con 0, nuestra variable de inicio marcaría que la primera vuelta es 0, por eso es importante que cuando se muestre le sumemos 1 como en el ejemplo.

```
Repetición número 1
Repetición número 2
Repetición número 3
Repetición número 4
Repetición número 5
Fin del programa
```

6.2.1. Uso de break para detener ciclo

Si queremos **detener** el **for** antes de que termine, usamos **break**.

```
for x in range (10):
    if x == 5:
        break
    print("Número de vuelta", x + 1)

print("Fin del programa")
```

De esta forma, con la estructura condicional que tenemos dentro del ciclo for se evalúa que número de vuelta es, cuando llegue x sea igual a 5 finaliza el ciclo.

```
Número de vuelta 1
Número de vuelta 2
Número de vuelta 3
Número de vuelta 4
Número de vuelta 5
Fin del programa
```

Recordar que cuando le sumamos 1 a x no alteramos su valor, solo se muestra incrementado pero su valor real sigue siendo 1 unidad menos.

6.2.2. Uso de continue para saltar una vuelta

Si queremos **saltar** una repetición sin detener el ciclo, usamos **continue.**

```python
for x in range (6):
    if x == 2:
        continue
    elif x == 4:
        continue
    print("Vuelta número", x + 1)
```

En este caso, si x vale 2 o 4 las vueltas no se mostrarán por pantalla, pero al sumar 1 unidad a x al mostrarse serán las vueltas 3 y 5:

```
Vuelta número 1
Vuelta número 2
Vuelta número 4
Vuelta número 6
```